

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1101

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

I Mathusri P-192311363(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mathusri P

Case study

QUESTIONS: Any 4

Total Marks:40

1. Online Hotel Reservation System

A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions.

Question:

Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

(10 Marks)

2. University Management System

A university intends to automate its operations including student admissions, course registration, examination management, and result processing. Different users such as students, faculty members, examination staff, and administrators interact with the system.

Question:

Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

(10 Marks)

3. Online Retail Shopping Platform

A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory.

Question:

Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.

(10 Marks)

4. Smart Waste Management System

A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application.

Question:

Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why.

(10 Marks)

5. Smart Healthcare Management System

A multi-specialty hospital plans to develop a Smart Healthcare Management System where patients can book appointments, consult doctors, access medical records, and make online payments. Doctors update patient diagnoses and prescriptions, while hospital administrators manage departments, staff, and billing information.

Question:

Analyze the system and explain how **object-oriented principles (encapsulation, inheritance, and polymorphism)** can be applied. Also, recommend a suitable **SDLC model** for developing this healthcare system and justify your choice.

(10 Marks)

6. Smart Library Management System

A university library intends to automate its operations, allowing students and faculty members to search books, borrow and return books, reserve titles online, and pay overdue fines. Librarians manage book inventories, memberships, and circulation records, while administrators generate reports and maintain the system.

Question:

Design a structured solution using **object-oriented concepts** for this system. Identify the major classes, attributes, methods, and relationships, and recommend an appropriate **SDLC model** for efficient development with suitable justification.

(10 Marks)

RUBRICS FOR CALCULATION:

Case study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Online Hotel Reservation System

Introduction and System Analysis

The Online Hotel Reservation System is designed to automate the process of searching hotels, checking room availability, booking rooms, making online payments, and providing reviews. The system allows customers to search for hotels based on location, room type, price, and availability. After selecting a suitable room, the customer can make a reservation and complete the payment through an online payment facility. Customers can also cancel bookings and provide reviews based on their experience.

Hotel managers are responsible for maintaining hotel information, updating room availability, changing room prices, and managing reservations. Administrators have higher-level responsibilities such as managing users, monitoring bookings, verifying transactions, and generating reports. Therefore, the system contains different types of users with different responsibilities, making object-oriented modelling suitable.

Application of Encapsulation

Encapsulation is an important object-oriented principle in which data and the operations performed on that data are combined within a class. It also prevents unauthorized direct access to important information. In the hotel reservation system, classes such as Customer, Room, Booking, and Payment can use encapsulation.

For example, the Room class can contain room number, room type, price, and availability status as private attributes. These values should not be directly changed by customers. Instead, methods such as `checkAvailability()`, `updatePrice()`, and `updateAvailability()` can be provided. Similarly, payment information can be protected inside the Payment class. This improves security and prevents invalid changes to the system.

Application of Inheritance

Inheritance allows a new class to acquire common properties and behaviours from an existing class. In this system, a general User class can be created containing common attributes such as user ID, name, email, password, and common methods such as `login()` and `logout()`.

The classes Customer, HotelManager, and Administrator can inherit from the User class. A customer can perform operations such as searching hotels and booking rooms, whereas a hotel manager can update room information. An administrator can manage users and transactions. Inheritance reduces code duplication and improves reusability.

Application of Polymorphism

Polymorphism allows the same operation to have different implementations depending on the object. In the hotel reservation system, different payment methods can implement a common `makePayment()` operation. For example, `CreditCardPayment`, `UPIPayment`, and `NetBankingPayment` can all implement `makePayment()` differently.

When the customer selects a payment method, the system can call the same payment operation without needing to know the internal implementation. This makes the system flexible and makes it easier to introduce new payment methods in the future.

Important Classes and Relationships

The major classes include User, Customer, HotelManager, Administrator, Hotel, Room, Booking, Payment, and Review. A hotel contains several rooms, so Hotel and Room have a composition relationship. A customer can make many bookings, while each booking belongs to a particular customer. A booking is associated with a payment, and a customer can provide reviews for hotels.

The inheritance relationship exists between User and its subclasses such as Customer, HotelManager, and Administrator. These relationships make the system easier to understand and maintain.

Suitable SDLC Model – Agile

The **Agile SDLC model** is suitable for developing the Online Hotel Reservation System because requirements in online travel applications can change frequently. Customers may expect new features such as promotional offers, cancellation facilities, loyalty programs, new payment methods, and personalized hotel recommendations.

Using Agile, the complete system can be divided into smaller iterations. The first iteration can implement registration and hotel searching. The second iteration can implement room booking, followed by payment, cancellation, reviews, and administration modules. Each iteration can be tested and demonstrated to stakeholders.

Agile also allows customers and hotel managers to provide feedback throughout development. Errors can be identified early and new requirements can be incorporated without redesigning the entire system.

Conclusion

Object-oriented principles provide security through encapsulation, reuse through inheritance, and flexibility through polymorphism. The system can therefore be developed as a collection of independent and reusable classes. Agile is the most suitable SDLC model because it supports changing requirements, continuous testing, frequent feedback, and incremental delivery.

2. University Management System

Introduction and System Analysis

The University Management System is designed to automate major academic and administrative activities of a university. The system handles student admissions, course registration, examination scheduling, marks entry, result processing, faculty management, and report generation.

The main users are students, faculty members, examination staff, and administrators. Students use the system to apply for admission, register for courses, view examination schedules, and access results. Faculty members manage courses, enter marks, and view student information. Examination staff manage examinations and verify marks, while administrators manage students, faculty members, departments, and courses.

Because the system contains many entities with their own data and behaviours, object-oriented analysis provides a suitable approach.

Object-Oriented Modelling

The major classes in the system include User, Student, Faculty, Course, Examination, Result, Administrator, and ExaminationStaff.

The Student class may contain attributes such as student ID, name, department, email, and year of study. Its methods may include applyAdmission(), registerCourse(), and viewResult(). The Faculty class may contain faculty ID, name, department, and specialization, with methods such as manageCourse() and enterMarks().

The Course class contains course ID, course name, credits, and department. A student can register for multiple courses, and one course can have many students. Therefore, the relationship between Student and Course is a many-to-many association.

Encapsulation

Encapsulation is useful for protecting sensitive university information. For example, student marks and grades should not be directly accessible to every user. The Result class can store marks and grades as protected or private data.

Methods such as enterMarks(), calculateGrade(), and publishResult() can be used to control access to result information. Only authorized faculty or examination staff should be allowed to modify marks. This prevents unauthorized modification and improves data security.

Inheritance

A common User class can contain common information such as user ID, name, email, password, and login functionality. Student, Faculty, ExaminationStaff, and Administrator can inherit from this class.

Each subclass can add its own specific behaviour. For example, students can register for courses, faculty can enter marks, examination staff can manage examinations, and administrators can manage the complete system. This approach improves code reuse and reduces duplication.

Polymorphism

Polymorphism can be applied when different users perform similar operations in different ways. For example, the method generateReport() can be implemented differently for students, faculty members, and administrators.

A student may receive a personal academic report, a faculty member may receive a course performance report, and an administrator may receive a university-level report. The same method name can therefore produce different results depending on the object.

Suitable SDLC Model – Agile

The **Agile model** is suitable because a university management system consists of several independent modules and requirements may change according to university policies.

The development can be divided into iterations. Admission can be developed first, followed by course registration, examination management, result processing, and reporting. Each module can be tested and reviewed before the next module is developed.

Agile also allows students, faculty, and administrative staff to provide feedback. If the university introduces a new grading system or changes its registration procedure, the system can be modified during a future iteration.

Conclusion

Object-oriented analysis provides a clear structure for the university system by identifying classes, attributes, methods, and relationships. Encapsulation protects sensitive information, inheritance promotes reuse, and polymorphism provides flexibility. Agile is suitable because it supports incremental development, continuous testing, and changing university requirements.

3. Online Retail Shopping Platform

Introduction and System Analysis

The Online Retail Shopping Platform is an e-commerce system that allows customers to browse products, add products to a shopping cart, place orders, make online payments, and track deliveries. Vendors use the system to add and update products, manage prices, and monitor stock. Administrators manage users, transactions, inventory, and overall system operations.

The system is large and contains several independent functional areas. Therefore, modular software design using object-oriented techniques is highly suitable.

Modular Object-Oriented Design

The system can be divided into different modules such as Customer Management, Product Management, Shopping Cart, Order Management, Payment Management, Delivery Management, and Administration.

The Customer class can contain customer ID, name, email, and address. It can provide methods such as `browseProduct()`, `addToCart()`, `placeOrder()`, and `trackOrder()`.

The Product class can contain product ID, product name, price, category, and stock quantity. Methods such as `updatePrice()` and `updateStock()` can be used to manage product information. The Order class can contain order ID, order date, total amount, and order status. Methods such as `createOrder()`, `cancelOrder()`, and `trackOrder()` can manage orders.

Encapsulation

Encapsulation allows each module to control its own data. For example, product stock should not be directly changed by customers. The Product class can provide `updateStock()` to modify inventory.

Similarly, payment information can be protected inside the Payment class. Only authorized operations such as `processPayment()` and `verifyPayment()` should be allowed. This reduces the possibility of incorrect or unauthorized data modification.

Inheritance

A general User class can contain common properties such as user ID, name, email, password, and login functionality. Customer, Vendor, and Administrator can inherit from the User class. This allows common functionality to be reused. Each subclass can then define its own operations. For example, a customer places orders, a vendor manages products, and an administrator monitors transactions.

Polymorphism

Polymorphism can be applied to the payment module. A common `processPayment()` operation can be implemented by different payment classes such as `UPIPayment`, `CreditCardPayment`, `DebitCardPayment`, and `NetBankingPayment`.

The order system only needs to call `processPayment()`. The selected payment class determines how the actual transaction is processed. This makes the payment module flexible and extensible.

Evaluation of SDLC Models

The Waterfall model provides a structured development process, but it is less suitable because e-commerce requirements can change frequently. Once a phase is completed, making major changes becomes difficult.

The Spiral model provides strong risk management and is useful for complex systems, but it can require more time and resources than necessary for normal e-commerce development.

The Agile model supports iterative development, continuous testing, customer feedback, and changing requirements. Therefore, Agile is the most appropriate model for this system.

Justification for Agile

The e-commerce system can be developed in small iterations. The first iteration can provide product browsing, followed by shopping cart and order management. Payment and delivery tracking can be added in later iterations.

Agile allows the company to introduce new payment options, discounts, recommendation features, delivery services, and vendor facilities based on market requirements.

Conclusion

Object-oriented modular design divides the large e-commerce system into manageable and reusable components. Encapsulation protects data, inheritance supports reuse, and polymorphism provides flexibility. Among the SDLC models, Agile is the most appropriate because of changing customer requirements, continuous feature development, and the need for frequent releases.

4. Smart Waste Management System

Introduction and System Analysis

The Smart Waste Management System is an IoT-based system developed to improve waste collection efficiency. Smart bins are equipped with sensors that continuously monitor the amount of waste inside them. When the waste level reaches a particular limit, the system generates an alert and notifies the appropriate collection vehicle.

Citizens can use a mobile application to report overflowing bins, illegal dumping, or other waste-related problems. Administrators can monitor bin status, assign collection vehicles, view reports, and manage the entire system.

The system involves hardware, software, sensors, mobile applications, communication networks, and real-time data. Therefore, object-oriented system development is suitable.

Major Classes

The important classes include SmartBin, Sensor, CollectionVehicle, Citizen, WasteReport, Alert, and Administrator.

The SmartBin class can contain bin ID, location, capacity, waste level, and status. Methods can include measureWasteLevel(), updateStatus(), and sendAlert().

The Sensor class contains sensor ID, sensor type, and sensor reading. The method readData() obtains information from the physical sensor.

The CollectionVehicle class contains vehicle ID, driver details, location, and status. Its methods include receiveAlert(), collectWaste(), and updateStatus().

Encapsulation

Sensor readings and bin information should be protected from unauthorized modification. For example, waste level information can be stored inside the SmartBin class and updated through measureWasteLevel().

Citizens should not be allowed to directly modify the status of a bin. Instead, the system should receive sensor readings or verified administrator updates. This ensures data integrity.

Inheritance

A general Sensor class can be created containing common attributes and methods. Different sensor types can inherit from it.

For example:

Sensor

→ UltrasonicSensor

→ WeightSensor

→ TemperatureSensor

Each type of sensor has its own specific function while sharing common sensor properties.

Polymorphism

Polymorphism can be applied through the readData() method. An ultrasonic sensor can use readData() to measure distance and calculate waste level, while a weight sensor can use the same method to measure the weight of waste.

The system can therefore work with different sensors through a common interface without changing the main waste-management logic.

Relationships

A smart bin contains one or more sensors, so the relationship between SmartBin and Sensor can be represented using composition. A citizen can create multiple waste reports. Administrators can monitor smart bins and assign collection vehicles.

When a bin becomes full, the SmartBin generates an alert, and the collection vehicle receives the notification. This provides an automated relationship between sensor data and waste collection.

Suitable SDLC Model – Spiral Model

The **Spiral Model** is highly suitable for this system because it combines iterative development with risk analysis. The project contains several technical risks related to sensor accuracy, network connectivity, real-time communication, hardware compatibility, and mobile application integration.

In the first cycle, a prototype can be developed to test the sensor and smart bin. The next cycle can integrate the sensor with the server. Another cycle can implement automatic vehicle notifications, followed by the citizen mobile application.

At each cycle, technical risks can be identified, tested, and reduced before proceeding further. This is especially useful because IoT requirements may evolve after real-world testing.

Conclusion

Object-oriented design provides a structured architecture for smart bins, sensors, vehicles, citizens, alerts, and administrators. Encapsulation protects sensor and system data, inheritance supports different sensor types, and polymorphism allows different sensors to operate through common methods. The Spiral Model is suitable because it supports iterative development, continuous risk analysis, prototyping, and evolving requirements.

5. Smart Healthcare Management System

Introduction and System Analysis

The Smart Healthcare Management System is designed to automate healthcare services in a multi-specialty hospital. Patients can register, book appointments, consult doctors, access medical records, receive prescriptions, and make online payments. Doctors can view patient information, enter diagnoses, and update prescriptions. Administrators can manage departments, staff, billing, and hospital operations.

Since the system handles sensitive medical information and involves different types of users, object-oriented analysis provides a clear way to model the system.

Major Classes

The important classes include User, Patient, Doctor, Appointment, MedicalRecord, Prescription, Payment, Department, and Administrator.

The Patient class can contain patient ID, name, age, address, and contact details. Its methods may include bookAppointment(), viewMedicalRecord(), and makePayment().

The Doctor class can contain doctor ID, name, specialization, and department. Its methods include viewPatient(), addDiagnosis(), and createPrescription().

The Appointment class contains appointment ID, date, time, doctor, patient, and appointment status. Methods such as book(), cancel(), and reschedule() can be included.

Encapsulation

Encapsulation is particularly important in healthcare because medical records and prescriptions contain sensitive information. Patient diagnosis, treatment details, and prescriptions should not be directly accessible or modified by unauthorized users.

The MedicalRecord class can keep diagnosis and treatment information private and provide controlled methods such as addDiagnosis() and updateRecord(). Access can be restricted so that only authorized doctors can modify the record.

Similarly, payment details can be encapsulated inside the Payment class and accessed through controlled operations such as processPayment() and verifyPayment().

Inheritance

A general User class can contain common information such as user ID, name, email, password, and login functionality.

The classes Patient, Doctor, Administrator, and Receptionist can inherit from User. Each subclass can have additional attributes and behaviours.

For example, a patient can book appointments, a doctor can update diagnoses, and an administrator can manage departments and staff. This avoids repeating common user functionality.

Polymorphism

Polymorphism can be applied to operations such as viewDashboard(). A patient dashboard can display appointments and prescriptions, while a doctor dashboard can display assigned patients and consultations. An administrator dashboard can display departments, staff, and billing information.

Similarly, different payment methods can implement a common processPayment() method. This allows the system to support multiple payment mechanisms without changing the main billing module.

Relationships

A patient can book many appointments, and a doctor can have many appointments. A patient can have medical records, and a doctor can update those records. A medical record may contain multiple prescriptions.

The User class has inheritance relationships with Patient, Doctor, Administrator, and Receptionist. The Patient and Doctor classes are associated with Appointment.

Suitable SDLC Model – Spiral Model

The **Spiral Model** is recommended because healthcare software involves high risks related to privacy, security, reliability, and correctness. Medical data must be protected, and incorrect system behaviour can have serious consequences.

Spiral development allows the project team to identify and address risks in every development cycle. Security requirements, access control, medical-record protection, payment security, and system reliability can be tested continuously.

The system can be developed incrementally, starting with patient registration and appointments, followed by medical records, prescriptions, billing, and administration.

Doctors and hospital staff can provide feedback during each cycle. Any new hospital requirement can then be incorporated into the next cycle.

Conclusion

Object-oriented principles make the healthcare system secure, reusable, and maintainable. Encapsulation protects medical and financial data, inheritance promotes reuse among different user types, and polymorphism provides flexible behaviour. The Spiral Model is appropriate

because it supports iterative development, continuous risk analysis, security testing, and changing healthcare requirements.

6. Smart Library Management System

Introduction and System Analysis

The Smart Library Management System automates the activities of a university library. Students and faculty members can search for books, borrow and return books, reserve unavailable books, and pay overdue fines. Librarians manage book inventory, memberships, circulation records, and reservations. Administrators generate reports and maintain the overall system.

The system contains several real-world entities such as users, books, memberships, reservations, circulation records, and fines. These entities can naturally be represented as objects and classes.

Major Classes, Attributes and Methods

The User class can contain user ID, name, email, and password. Its methods include login() and logout().

The Student class can contain student ID and department. It can provide methods such as searchBook(), borrowBook(), returnBook(), and reserveBook().

The Faculty class can contain faculty ID and department and can also borrow, return, and reserve books.

The Book class contains book ID, title, author, ISBN, category, and availability status. Its methods include search(), checkAvailability(), and updateAvailability().

The CirculationRecord class contains issue date, return date, and status. Its methods include issueBook() and returnBook().

The Reservation class stores reservation ID, reservation date, and status. It provides methods such as reserveBook() and cancelReservation().

The Fine class contains fine ID, amount, and payment status. Its methods include calculateFine() and payFine().

The Librarian class manages books, members, and circulation records. The Administrator class generates reports and manages system settings.

Encapsulation

Encapsulation can be used to protect important library information. For example, the availability status of a book should not be directly modified by a student. Instead, the Book class can provide controlled methods such as checkAvailability() and updateAvailability().

Similarly, fine information should be protected within the Fine class. The amount can be calculated using calculateFine() based on the number of overdue days. The user can pay the fine using payFine().

This approach prevents invalid modifications and improves system security.

Inheritance

A general User class can contain common details such as user ID, name, email, and login functionality.

The classes Student, Faculty, Librarian, and Administrator can inherit from User. Each class can then implement its own specific functions.

For example, students and faculty members can borrow books, librarians can issue and receive books, and administrators can generate reports. This reduces duplicate code and improves maintainability.

Polymorphism

Polymorphism can be applied to borrowing rules. A common method such as `getBorrowLimit()` can be implemented differently for different users.

For example, a student may have a lower borrowing limit, while a faculty member may have a higher limit. The same method can therefore provide different results depending on the user object.

Polymorphism can also be used for notifications, where email notification and mobile notification can implement a common `sendNotification()` operation.

Relationships

The User class has an inheritance relationship with Student, Faculty, Librarian, and Administrator.

A student or faculty member can borrow multiple books. A book can have many circulation records over its lifetime. A user can create reservations for books. A circulation record may generate a fine when a book is returned late.

The Librarian is associated with books and circulation records because librarians manage issuing and returning books.

Suitable SDLC Model – Agile

The **Agile SDLC model** is suitable for the Smart Library Management System because the system can be developed incrementally and library requirements may change over time.

The first iteration can implement user registration and login. The second iteration can implement book search and inventory management. The third iteration can implement borrowing and returning. Later iterations can implement reservations, fine payments, reports, and notifications.

Agile provides frequent feedback from librarians, students, and faculty members. If the library later introduces barcode scanning, QR-based book identification, mobile notifications, or digital books, these features can be added in future iterations.

Agile also supports continuous testing, allowing each module to be verified before the complete system is deployed.

Conclusion

The Smart Library Management System can be effectively designed using object-oriented concepts. Classes represent real-world entities, attributes store their data, methods represent their behaviour, and relationships describe their interaction. Encapsulation protects important information, inheritance promotes reuse, and polymorphism provides flexible behaviour. Agile is the most suitable SDLC model because it supports incremental development, continuous feedback, testing, and future enhancements.